Master Guide: Data Cleaning Techniques in PySpark

A Comprehensive Study Resource for Car Sales Data Engineering

---

1. Handling Null Values (Imputation)

Data often arrives with missing values. Instead of deleting records, we "fill" them to maintain data integrity.

Technical Implementation

- **`.fillna(value)`**: Replaces all nulls in the DataFrame with a specific value.
- **`.fillna({dict})`**: Replaces nulls in specific columns with specific values (Best Practice).

Detailed Use Case (Car Sales)

In your `customers` table, many users might skip entering their **City** or **Name**.

- **Problem:** If you perform an Inner Join on a table with null names, your reports will look unprofessional (showing "null" in charts).
- **Solution:** Replace nulls with `"Unknown"`.

```python
# Strategy: Keep the row, but provide a placeholder
customers_cleaned = customers_df.fillna({"name": "Unknown", "city": "Not Specified"})
```

---

2. Fixing Negative Values (Sign Correction)

Negative numbers in columns like `Price`, `Tax`, or `Quantity` are almost always data entry errors.

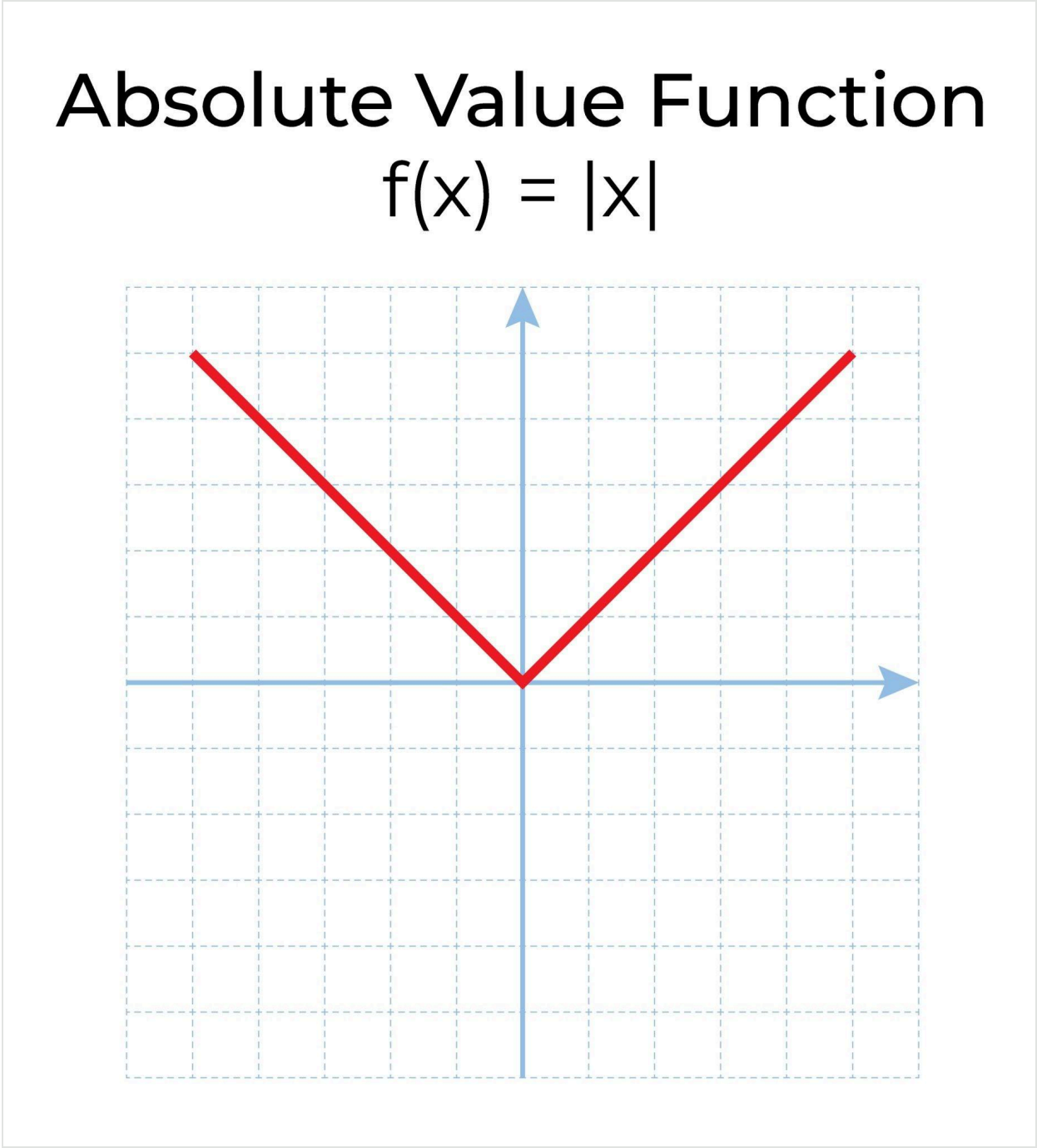
Technical Implementation

- **`abs(col)`**: Returns the absolute (positive) value of a number.
- **`when().otherwise()`**: A conditional statement (If/Then/Else) used to set a floor (e.g., if price < 0, set to 0).

Detailed Use Case (Car Sales)

A car dealership might accidentally list a trade-in as a negative price in the `cars` table (e.g., `-15000`).

- **Problem:** Summing these values will result in an incorrect (lower) Total Revenue.
- **Solution:** Use the absolute value to ensure every car has a positive price tag.



-
- Getty Images

Python

```python
from pyspark.sql.functions import abs, col

# Strategy: Assume the number is correct, but the sign is wrong
cars_cleaned = cars_df.withColumn("price", abs(col("price")))
```

---

3. String Standardization (Trimming & Formatting)

Leading or trailing spaces are "invisible" killers in data engineering. "Honda" is NOT the same as " Honda".

Technical Implementation

- **trim(col)**: Removes spaces from both ends.
- **ltrim / rtrim**: Removes spaces from the left or right only.

Detailed Use Case (Car Sales)

When joining the Sales table to the Cars table using car_id.

- **Problem:** If the car_id in Sales is " 101" but the ID in the Cars table is "101", the **Join will fail**, and you will lose that sales record in your final report.
- **Solution:** Trim all ID and Category columns (Brand, Model, City) before joining.

Python

```python
from pyspark.sql.functions import trim

# Strategy: Clean 'Key' columns to ensure Join success
sales_cleaned = sales_df.withColumn("car_id", trim(col("car_id")))
```

---

4. Deduplication (Removing Invalid Duplicates)

Duplicate records occur when data is re-sent or processed multiple times.

Technical Implementation

- **.dropDuplicates([subset])**: Removes rows based on specific columns.
- **Window Functions**: Used for "Tie-breaking" when you have two records for the same ID and need to keep the "best" one.

Detailed Use Case (Car Sales)

In the sales table, you might find the same sale_id twice because the system glitched.

- **Problem:** Counting both rows would double your "Units Sold" and "Total Revenue," leading to false business insights.
- **Solution:** Drop duplicates based on the primary key (sale_id).

Python

```python
# Strategy: One Sale ID must represent exactly one row
sales_final = sales_cleaned.dropDuplicates(["sale_id"])
```

Summary Table for Self-Study

Technique	Function	Best For	Impact on Data
Fillna	df.fillna()	Missing Names/Cities	Row count stays same.
Absolute	abs(col)	Negative Prices	Values corrected; count stays same.
Trim	trim(col)	Hidden spaces in IDs	Vital for Join accuracy.
Drop Dups	dropDuplicates()	Repeated Sales IDs	Row count decreases; accuracy increases.